Lambe User Guide

Android and Windows app • Version 0.9.6 • Open alpha

Lambe lets you transcribe text into Tengwar, adjust individual letters and marks, create your own writing conventions, and combine selected shapes into a typeable font glyph. This guide begins with your first transcription and then explains the tools for editing, mode creation and font work.

Tengwar is a writing system created by J.R.R. Tolkien. A script is different from a language: English written in Tengwar is still English. Lambe transcribes your input; it does not translate it into Quenya or Sindarin. Choose a mode for the language and convention you intend to write.

The terms you need

A tengwa is a base letter; tengwar is the plural. A tehta is an added mark or diacritic; tehtar is the plural. Some tehtar represent vowels, while others indicate doubling, nasalisation, a final form or another feature. A tehta is not automatically a vowel.

A carrier gives a mark a base when there is no suitable letter to receive it. A mode defines how input becomes letters and marks. A font defines their shapes. Changing a mode changes the writing convention; adjusting a shape or its position changes the particular composition.

How to use this guide

Begin with Installing and updating Lambe and First transcription. Continue through modes, positioning, saving and custom definitions. Arabic romanised input explains the included Arabizi profile. The final chapters cover fonts, practical help and an optional reference for exported mode files.

You can begin without programming knowledge. Use the graphical controls and glyph libraries first; the file reference is optional.

Created and developed by André I. Ben. Published by Lambe Labs.

Installing and updating Lambe

Download the file for your device from the Lambe release page. Windows uses a ZIP containing the complete app. Android uses an APK that installs the app. The user guide and Arabic mode JSON are also available separately.

Windows

Extract the complete Windows ZIP before opening it. Keep Lambe.exe, the DLL files, data, modes and the accompanying folders together. Open Lambe.exe from the extracted folder. A shortcut may point to that executable. You can place the whole folder in another writable location without changing mode references.

Do not launch the app from inside the ZIP or copy only the executable. The data folder contains the bundled fonts and definitions. The modes folder holds the portable mode collection when it is writable. The app can use Windows app data if the installation folder does not permit saving.

Android

Download the APK to your device and open it. If Android asks, allow installation from the browser or file manager used to open that download. The package includes its fonts and modes. Android stores saved definitions within the app; use Export mode to make a copy you can see and share.

Updating and moving your work

Close Lambe before replacing files. On Windows, extract a new release into a new folder. Copy your previous modes/custom_modes.json and, if present, its .bak file into the new modes folder before launching. You may also export individual modes and import them in the new installation. Built-in definitions are supplied by each release.

Install a later Android release over the previous one when Android permits it. Export custom modes first if an older development build must be removed. Uninstalling or clearing app storage can remove saved definitions. Keep the source text, images and fonts you want before closing the current session.

First transcription

Open the installed app. The main input starts blank. You do not need to remove an example sentence before entering your own text.

1. Choose Writing mode in 1. Source input. For an English phrase, select English. Other languages need their appropriate modes.

2. Click or tap the input box and type a short phrase, such as “the garden”. The result appears automatically in 2. Tengwar output.

3. Read the result before changing it. Shorthand and combined sounds can mean that several input letters produce one tengwa, or that a vowel becomes a mark on another letter.

4. Select a base or mark on the canvas. If the shapes overlap, use the labelled choices beneath the output to select the exact object.

5. Move the selected object slightly with the direction pad, or drag it. A base and its tehtar can be selected and moved independently.

6. When the result is ready, choose Export image and save the PNG. Exporting the image does not erase your current composition.

Entering and navigating text

Use Enter to create a new line. Blank lines and a final empty line are retained in the result. The input and output grow vertically; long output lines can be scrolled horizontally. Windows uses the available window width and provides a wider page scrollbar.

Plain arrow keys move the text caret. Home, End and page-navigation keys also support editing. Hold Shift when you intend to select text. Click back into the input before typing; the physical number pad controls positioning when the canvas is selected, rather than when you are editing the input.

Numbers

The tengwar numerals switch starts off. Leave it off to keep 0-9 as ordinary digits, or turn it on to use the bundled Tengwar digit shapes. The switch updates the result while retaining its positioning. It preserves decimal digit order and leading zeroes; it does not convert numbers into base twelve.

Try “room 007” with the switch off and on. You should see the same three digits represented in different shapes.

Choosing a writing mode

Writing mode controls transcription. Select the mode before making detailed corrections, because changing the mode regenerates the text-derived objects. The same input can produce different valid arrangements under different conventions.

Included modes

English provides an orthographic starting point with conventional shorthand, combined consonants and pronunciation-sensitive readings. Hebrew is the bundled Romanised Hebrew convention. Quenya, Sindarin, Sindarin Mode of Beleriand, German Orthographic and Arabic · Arabizi (McGovern-based) are also included.

Quenya normally places vowel tehtar on the preceding tengwa. Sindarin and German normally use the following tengwa. Beleriand uses full letters for vowels. These conventions affect where a vowel belongs; dragging a mark is a visual adjustment, not a change to the mode's underlying convention.

The bundled Hebrew mode expects its Romanised input convention. Hebrew-script input can be supported by custom associations and rules. A dual-input Hebrew mode must contain the required mappings; selecting the bundled Hebrew mode does not by itself install a custom Hebrew-script definition.

Reading the English result

The English mode is a starting point rather than a pronunciation authority. Names, loans, unusual spellings and personal conventions may need correction. For recurring changes, create a custom mode instead of repairing every occurrence manually.

NT uses tinco with a nasal bar above. MP uses parma with that bar. Hard NC and NK use quessë with a nasal bar; NC before E, I or Y retains a separate N when C is treated as soft. QU, QW and KW use quessë with a w curl above.

Final S can be represented by a curl on the preceding letter. A final S followed by * requests a separate S letter in the English mode. In version 0.9.6 all final-S curl variants have a default adjustment of 0.2 units to the right; your own movement remains additional to that adjustment.

Language and appearance

Choose a different mode when the language or convention differs. Use the selected-object controls when one glyph or position needs changing. Importing a font into the font panel prepares a font for editing; it does not switch the main transcription canvas to that typeface.

Arabic romanised input

Choose Arabic · Arabizi (McGovern-based) in Writing mode. The mode is included from the first launch and its input starts blank. It transcribes romanised Arabic into Tengwar. It does not translate Arabic or reconstruct Arabic script from a dialectal spelling.

Digits used as letters

The profile uses 2 for hamza and 3 for ʿayn. Write 3' or gh for ghayn. Write 7 for ḥāʾ, and 5, 7' or kh for khāʾ. Write 6 for ṭāʾ and 6' for ẓāʾ. Write 8 or q for qāf, 9 for ṣād, and 9' for ḍād. Straight and curly apostrophes work in these combinations.

A standalone apostrophe or ʾ represents hamza. The characters ʿ and ʕ also represent ʿayn. Input is case-insensitive, so use 9 to distinguish ṣād from sīn; capital S does not create that distinction.

Other consonants

The consonant spellings are b, t, th, j, d, dh, r, z, s, sh, f, k, l, m, n, h, w and y. The spelling dj also represents jīm, and x represents khāʾ. Academic forms including ḥ, ṣ, ḍ, ṭ, ẓ, ġ, ḫ, ṯ and ḏ are accepted.

The letters g and the combination ch remain literal text until you assign their intended sound in your own copy. Arabizi conventions differ across communities. This profile assigns 8 to qāf and 9 to ṣād; create an independent copy if your convention differs. A dialectal glottal stop written 2 is treated as that pronounced sound.

Making your own Arabic convention

Use Create your own mode and choose the existing Arabic mode as the starting point to make an independent copy. Change its associations, contextual rules or vowel settings, then save it. The separate arabic-romanized-lambe.json file can also be imported as a new editable copy.

Arabic vowels and literal numbers

Vowels and doubling

Short a uses three dots above the preceding consonant. Short i or e uses a dot above it; short u or o uses a right curl. A short vowel at the beginning of a word receives its own carrier. The e and o spellings are convenience aliases for the i and u vowels in this profile; they do not define separate dialectal vowels.

Write aa, ā or â for long a. Write ii, ee, ī or î for long i, and uu, oo, ū or û for long u. Use w for a consonantal w and y for a consonantal y. Write the diphthongs as aw and ay.

Repeat a complete consonant input for doubling: bb, ll, shsh, khkh or 3'3'. The result is one consonant with a doubling bar below. In mu7ammad, the doubled m has this bar.

Optional explicit spellings

Use t* for tāʾ marbūṭa, a* for alif maqṣūra, and a~, i~ or u~ for tanwīn. These are explicit input choices; the app does not infer them from endings. The spellings laa and lā use the lam–alif form. Hamza uses an independent calma throughout this profile.

Writing an actual number

In Arabic mode, an unescaped digit with an association is a letter. Put one backslash before an entire digit run when you mean an actual number: \2026 or \123. The backslash is an input control and is not drawn. The Tengwar numerals switch changes those number clusters, while 3 as ʿayn and 7 as ḥāʾ remain letters. Digits 0, 1 and 4 are numerals without an escape because this profile does not assign them to letters.

Try your own short examples such as 3arabi, 7abibi, 7abiibi, mu7ammad and salaam. Compare 3a \3 7 with the numeral switch off and on: only the escaped 3 changes its shape. Line breaks and empty lines are preserved.

Basis of the profile

The consonant assignments and basic vowel signs follow Gabriel McGovern’s 2016 Arabic Tengwar charts. The Arabizi aliases and input controls are Lambe adaptations. The profile does not claim to standardise Arabizi or reproduce every orthographic detail of the original chart. The Arabic guide accompanying the mode provides its source links.

Positioning letters and marks

Select the particular tengwa or tehta you want to move. Selection highlights the object. The labelled choices beneath the output help when a small mark is difficult to tap or several shapes occupy the same area.

Direction controls

Each direction-pad press moves 0.2 of a layout unit. On Windows, the physical number-pad digits use the same directions when the editor has canvas focus. The layout follows the number pad: 8 is up, 2 is down, 4 is left and 6 is right. The corner digits move diagonally; 5 is the centre control.

Relative movement adds each press to the current position. The switch in 3. Position selected tengwa or tehta changes this to Set position from centre, where a direction establishes a position rather than adding another step. Keep Relative movement for small repeated adjustments.

Dragging gives a free offset. Drag a tehta to move that mark only. Drag a base to move that base only; its marks remain independently positioned. For a base and marks to travel together, select them all using Select multiple.

Resetting and undoing

Clear position resets the selected object's editable placement. The centre control also resets position. Undo move / rotation reverses recorded position or quarter-turn changes. It is not a general undo for text edits, deletions or changes to a mode.

Use 90° left or 90° right for quarter-turn rotation. Rotation applies to the selected objects. Free-angle and 45-degree rotation are not controls in this release.

Android

Selecting an object opens the compact movement pad. Dismiss it with its close button and reopen it with the movement button. It hides while the text keyboard is open. Scroll the page to reach the full positioning and selected-object panels.

S and Z forms

For silme and esse families, 4. Selected object offers Form of S / SS / Z. Auto flips the form when an above mark needs it; Upright and Nuquerna choose a form explicitly. Nuquerna is the inverted form. A manual choice takes priority over automatic flipping.

A positional correction changes this occurrence. To make a recurring attachment or form choice part of future transcription, edit a copy of the mode.

Editing and composing glyphs

The output contains individual bases and marks. You can replace a generated shape, add a missing mark, remove an unwanted object or assemble a composition manually.

Changing a base

Select the tengwa, then use Tengwa library in 4. Selected object to choose a replacement. The Base token field is an alternative for users who know the catalogue name. A token is the app's name for a glyph, such as parma or lambe; it is not text intended to appear in the result.

Delete base removes the selected base together with its marks. To remove only a tehta, select that mark and use Delete selected mark. Do not delete the base when your correction concerns just one attached mark.

Adding and changing marks

With a base selected, choose Browse tehtar to open the searchable mark library. Selecting a glyph adds it to the base. Add mark uses the name in Mark token for next/selected mark. If a mark is already selected, the Tehta library dropdown replaces that mark; if no mark is selected, it adds one.

A newly added mark begins at its default placement. Select it to move or rotate it. Above and below are positions, not declarations that the mark must be a vowel. The library includes marks for other writing features.

Adding a manual base

Base in the output heading adds a tinco as a starting object. Browse tengwar lets you choose a different base directly from a visual library. Search by the glyph's name or font address, then position it as required. Manual additions appear after the existing sequence.

Selecting several components

Enable Select multiple beneath the output. Select each base and each tehta that belongs to your composition. The selection count confirms how many objects are included. Selecting a base does not automatically select all its marks. Use Clear selection to begin again.

A multiple selection can move or rotate together and can be captured as one font glyph. Keep Select multiple off for ordinary single-object corrections.

Changing the source afterwards

Edits to the input can regenerate affected objects. Some adjustments to unchanged objects are retained, but a restructured word or a mode change can replace them. Finish the source first where possible, then make detailed positional corrections. Export an image before a large revision if you need a record of the current appearance.

Exporting and keeping your work

Exporting a picture

Choose Export image in 2. Tengwar output. Pick a filename and location in the save dialog. The exported PNG captures the complete result, including parts outside the visible width, explicit line breaks, empty lines and trailing empty lines.

The picture uses dark glyphs on a white background and excludes selection highlights. It is rendered at twice the canvas dimensions for a sharper image. A PNG preserves the appearance for sharing, printing or use in another design application, but its letters are not editable text.

The app has no programmed line-count limit. Very large images are still limited by the device's memory and graphics capabilities. If export fails, try a shorter section of the text and keep the original source separately.

Copyable records

Use the plus menu in the app bar to add Source + code, Document JSON or Position inspector. Each view has a copy button and a close button. Closing a view only removes that view from the screen.

Source + code opens a view headed Source + positioning code. It records the source, mode and named glyphs together with positions and rotations. This is a plain-text description of the composition, not a string that another word processor will automatically display as Tengwar.

Document JSON contains the document's source, generated and manual objects, positions, numeral setting and custom definitions. Copy it into a text file to keep a detailed record or include it with a problem report. Position inspector describes the currently selected object.

These views are read-only. Version 0.9.6 does not expose an Open document or paste-to-restore command for these document records. Do not treat them as an in-app save-and-reopen workflow.

What persists between sessions

Saved custom modes are kept locally and loaded when the app starts. On Windows, the app uses the modes folder beside Lambe.exe when that folder is writable. If that location is read-only, it uses your Windows app data. On Android, modes stay in private app storage. The supplied definitions are included with the app; you do not need to find them on the creator's computer or download them separately.

Export mode creates a separate .tengwar-mode.json file that can be shared or imported. When moving a portable Windows installation, move the whole folder, including modes. Export important custom modes before replacing an installation or uninstalling Android. The source, current composition and unsaved font data do not reopen automatically. Keep source text, export the appearance and save your font before closing the app.

Creating and duplicating modes

A custom mode makes recurring decisions part of transcription. You can change an existing convention, introduce shorthand, support another input script or start a definition from nothing.

1. Choose Create your own mode beside Writing mode.

2. In Start from, select Standard, Empty, a built-in mode, or one of your saved or imported modes. Choose Create.

3. Give the new definition a distinct Mode name. Review its associations, rules and switches.

4. Enter your own text in Try your mode. This preview starts blank and updates as you edit the definition.

5. Choose Save mode. The new mode becomes selected in the main editor and is saved locally. Use Edit mode to return to it.

Choosing a starting point

Standard starts with the English transcriber, including inherited spelling behaviour, shorthand and contextual rules. It is convenient when most English output is already suitable and you want specific exceptions.

Empty begins without associations or transcription rules. Unmapped input remains visible as literal text, which helps you identify gaps in the definition. General mode settings are still available; an enabled R rule does not create a missing R association for you.

Selecting an existing mode creates an independent copy with its own identifier and name. Its associations, attached marks, rules and enabled or deleted settings are retained. Editing the copy does not alter the original. Copies of supplied language modes also retain their inherited mapping and exception data.

Saving and sharing

Save association or Save rule confirms an individual edit inside the mode editor. Save mode commits the whole mode. If you leave with unsaved changes, the app offers to keep editing or discard them.

Export mode writes the selected custom definition to a .tengwar-mode.json file. Import mode reads this format and creates an editable copy with a fresh identifier. Import is not a silent replacement of an existing saved mode.

If the app reports that local saving failed, the definition may remain usable for the current session. Use Export mode immediately to retain it. The supplied JSONC language definitions and exported Lambe custom-mode files are different formats; Import mode expects the exported Lambe format.

Associations and combined output

An association connects literal input with an output. The input can be one character, a group such as sh, a Hebrew letter or a modifier sequence such as ב+. The meaning of a modifier is whatever your mode assigns to it.

Adding a base or mark

Choose Add tengwa or Add tehta. Enter Input text, choose the glyph and confirm with Save association. Tehta associations also have a Placement choice. Tap an existing association to edit it; Find an association helps locate one in a long list.

Input is matched without a case distinction. Use lowercase Latin input when authoring a definition; uppercase text can still be transcribed. Do not put spaces inside an association input. Longer inputs are matched before shorter associations at the same position, so sh can be distinguished from s followed by h.

One input with several components

A tengwa association can include several attached tehtar. Use Add tehta to association, choose each mark's placement and adjust its horizontal offset. This is useful for a shorthand form or a letter that always carries a distinguishing mark.

Add tengwa to association turns a tehta association into a combined output while retaining its mark. A single input can therefore produce a base, a mark or a base with several marks. The result remains editable as separate objects on the canvas.

Vowel classification

This input represents a vowel describes how the R rule should treat the input. It is independent of whether the output is a tengwa or a tehta. A full letter can represent a vowel; a diacritic can represent a consonantal feature and remain classified as a consonant.

For example, a mode can associate e with yanta as a full vowel letter. Mark e as a vowel even though the output is a tengwa. Conversely, a final-letter dot can be a tehta without being a vowel. These are illustrative configuration choices, not universal Tengwar assignments.

Disabling and deleting

Turn an association's switch off to disable it temporarily. Disabled association input is preserved as literal text rather than producing the assigned glyph. Delete removes the association and remembers the removal of an inherited default. You can add it again later.

Custom rules take precedence over associations at the same position. In Standard, a higher-level English combination can still control input such as mp. If changing p does not alter that combination, adjust its rule or disable the inherited rule group that supplies it.

Invisible input and the R rule

A No output association consumes its input without drawing a tengwa, tehta, carrier or extra space. It can simply remove a control symbol, or it can influence the R rule without changing the visible writing.

A marker for the preceding input

1. In your custom mode, choose Add no output and enter ^ as Input text.

2. Enable Classify the preceding input.

3. Enable Treat preceding input as a vowel.

4. Choose Save association, then Save mode. Keep the mode's R rule enabled if you want this classification to affect R selection.

For a mode containing r and b associations, rb^ lets the final marker classify b as a vowel for the R decision. R can consequently take the before-vowel form, while b keeps its normal output and ^ disappears. The marker does not move b or supply a visible vowel mark.

What the switches mean

Classify the preceding input targets the immediately preceding matched input association in the same word. That association may contain several characters, such as sh. The classification does not cross a space or punctuation boundary.

With Classify the preceding input on and the vowel switch off, the preceding input is treated as a consonant for the R rule. With preceding classification off but the vowel switch on, the invisible input itself can participate as a vowel in R look-ahead.

To consume a symbol without a classification effect, switch both classification options off. No output remains selected in Output.

Carrier syntax and silent syntax

In modes that enable explicit carriers, ^ normally inserts a carrier. A matching enabled No output association can give it your new silent meaning in that custom mode. This does not reassign ^ globally or change the behaviour of unrelated modes.

The classification belongs to R-rule management. It does not convert a consonant into a vowel tehta, change its glyph or automatically attach a mark. Use a visible association when you need an actual vowel sign.

No-output definitions are retained when a mode is saved, exported, imported or duplicated. Test the same word with and without the marker in Try your mode to check the difference.

Contextual rules and exceptions

Use Add rule when output depends on position or surrounding input. A rule can supply a tengwa, one or more tehtar, or a combined form. Custom rules match literal input inside a word.

Choosing the context

Match this input specifies the text to consume. Where can be Anywhere, Whole word, Start of word, End of word, Before input or After input. Before input checks the text immediately after the match. After input checks the text immediately before it.

For Before input and After input, enter the alternatives in Surrounding input, separated by commas, for example a,e,i,o,u. The context is checked but not consumed. Use lowercase alternatives for Latin input. One rule has one scope; it does not combine simultaneous before-and-after conditions.

Choosing what the rule produces

With Tehtar only and Add marks to preceding tengwa both off, choose Output tengwa and add any required marks with Add tehta to rule. Each mark has its own placement and horizontal offset.

Tehtar only produces marks without an explicit new base. They follow the mode's attachment direction, using a carrier when necessary. Add marks to preceding tengwa forces them onto the previous object; if there is no suitable previous object, a carrier is supplied.

A final-letter example

To add a dot below a final m, create a rule matching m at End of word, choose malta and add dotBelow with placement Below. Test m inside a word and at its end. The rule affects the final occurrence without changing your ordinary m association.

A final-letter convention for Hebrew can use one rule for each relevant input. For Romanised input, match the appropriate Latin letter at End of word; for Hebrew-script input, match the corresponding final letter. These are mode-author decisions, not changes to every bundled mode.

Which rule wins

At a source position, enabled custom rules are considered before associations. The longest matching rule wins; equal-length rules use their list order. A narrower context does not automatically outrank an equally long earlier rule. Put a specific equal-length exception before its general rule when editing the exported definition.

The output is not re-read as input. Graphical custom rules do not execute code or accept regular expressions. A disabled rule is skipped, allowing the other applicable rules or associations to handle the input. Use the bin button to delete a rule you no longer need.

Mode behaviour and inherited English rules

Attachment and carriers

Tehtar attach to the following tengwa controls the default direction of mark associations. Turn it off to attach to the preceding tengwa where possible. A rule that explicitly attaches to the preceding tengwa takes priority for that rule.

Carrier selects the base used when a mark has no suitable host. The ^ inserts an explicit carrier switch enables that input convention. An active No output association for ^ can override its visible effect within your custom mode.

Automatic forms

Automatically flip S / SS / Z allows silme and esse to use nuquerna forms when a mark sits above. An explicit Upright or Nuquerna choice for an individual object takes priority. The setting also applies to manual additions and captured font glyphs.

R rule chooses rómen before a sounded vowel and óre in the other English contexts, including final R. Custom associations can supply their own vowel classification. A silent marker can influence that classification without drawing anything.

Turn R rule off to keep your R association as specified. Its delete button removes the contextual R rule from that mode. To replace it, use ordinary custom rules with the contexts you require. This is useful when your writing convention uses different R forms or different bases entirely.

Inherited English controls

Standard and English copies expose switches for shorthand, silent E, repeated vowels, diphthongs, consonantal Y, labialisation, nasal combinations, TH, CH, GH, CK, doubled consonants, X, final S, soft C and voiced-S readings. Switch off a group when you intend to control that feature yourself.

A switch temporarily disables a group. A bin button deletes the inherited group from your definition. Deletions remain recorded when you save, duplicate or export the mode. The built-in English mode itself is unchanged.

The English silent-E behaviour includes the project’s convention for the whole word be: umbar with a dot below and no following carrier. This does not make every final E silent. You can change a particular word with a Whole word rule.

Testing a definition

Use Try your mode after each change. Compare ordinary input, modified input and the intended exception. Include a word with the same letters in another position to check that a contextual rule is not too broad. Test several lines and punctuation if they matter to your convention.

Save mode before returning to the main editor. If the result looks unchanged, confirm that your copy is selected in Writing mode and that a custom rule or inherited combination is not taking precedence.

Turning a selection into a font glyph

5. Selection to font glyph combines chosen vector shapes into one character. This can preserve a shorthand form, a custom letter or a carefully positioned base-and-mark combination for use in another text application.

1. Compose the shapes in the output. Enable Select multiple and select every base and tehta you want included. Check the selection count; attached marks are separate selections.

2. Choose New font for a small font containing your own glyphs, Import font to extend a suitable existing font, or Use bundled font to begin with the supplied Tengwar glyph set.

3. Enter Font family and Glyph name. A glyph name is an internal label, such as myShorthand; it is not the character you type. Use letters, digits, underscores or dots, beginning with a letter or underscore.

4. Enter Character code (hex). An unused private-use address such as E200 can hold a custom glyph. If you import a font, check that the selected address is free; the app can suggest a free address when the current one is occupied.

5. For a first attempt, keep Scale (%) at 100 and Baseline shift (font units) at 0. Leave Advance width (optional) blank for automatic spacing.

6. Choose Add selection as glyph. Inspect the preview rendered from the generated font. Add more selections at different character addresses if needed.

7. Choose Save font (.ttf), then save the file before closing the app or starting another font. Use Copy glyph character to copy the character you have assigned.

Using the result

Install the saved font through your operating system’s normal font controls, where supported, and select it in the text application you are using. Paste the copied character. A private-use character shows the intended glyph only when the matching font is available and selected.

Creating a glyph does not automatically add a word association to a Lambe mode. The exported character is for applications that can use the saved font. The main Lambe canvas continues to use its bundled glyph catalogue.

What is captured

The capture contains the selected outlines, their positions, quarter-turn rotations and chosen forms. It excludes selection decoration and editor-only spacing indicators. A PNG is a picture; this operation creates a font outline that can be scaled as text.

Adjusting and sharing fonts

Size and spacing

Scale (%) changes the selected composition's size in the font. Baseline shift (font units) adjusts its vertical placement relative to the writing line; positive values raise the glyph. These units belong to the font and differ from the canvas movement units.

Advance width is the horizontal distance reserved before the next character. It is not the visible width of the drawing. Leave it blank to calculate automatic side bearings, or enter a deliberate width for your convention. A composition containing two bases can become one wide character.

Existing characters

Leave Replace character mapping if it exists off when adding a new character. The app protects an occupied address rather than silently replacing it. Enable replacement only when you intend to assign that address to your new glyph.

Importing and saving creates a new font file; it does not overwrite the imported file automatically. Existing glyphs and character mappings are retained when you extend a supported font. Choose a distinct family name so the saved font can be distinguished from its source.

Supported imports

A static TrueType font is the most straightforward choice on both Android and Windows. Some .otf files also contain TrueType outlines. CFF OpenType files require conversion and may not import directly, especially on Android. If import fails, obtain a static TTF version of the font.

Variable fonts need a static instance first. Font collections and CFF2 fonts are not supported. A rejected file should be replaced with a supported version rather than assumed to have been partially imported.

Keeping a working font

Font work remains in the current session until saved. New font, another import or closing the app can discard the current working font. Save regularly with Save font (.ttf). Reimport the saved TTF when you want to add further glyphs in a later session.

Share the font alongside the characters or text that use it. The character alone does not tell another computer what its private-use shape should be. Copy glyph character copies one assigned character, not your source word or a complete mode.

Credits and licences

The bundled outlines come from Alcarin Tengwar by Toshi Omagari under the SIL Open Font License 1.1. The bundled TrueType copy is named Tengwar Lab Base. Retain the relevant licence and attribution when distributing fonts made from these outlines. For an imported font, follow that font's own licence.

Lambe retains the font notices it carries into supported exports. Give recipients the applicable licence information with the saved font; a custom family name does not remove the source font's licence.

Practical help and alpha feedback

When the result is unexpected

Wrong letters usually point to the selected mode, an association or a spelling rule. Correct letters in the wrong place point to attachment or positioning. Check Writing mode first, then inspect the base and each tehta separately.

If a drag moves the wrong object, use the labelled choices below the result. If a whole group moves unexpectedly, turn Select multiple off or use Clear selection. If the number pad types digits instead of moving a glyph, leave the input box and select the canvas object again.

If a custom change has no effect, check that the association or rule is enabled, that Save mode has been used and that your custom copy is selected. Review inherited combinations and longer input matches. An Empty mode showing Latin or Hebrew characters often has an unmapped input, rather than a rendering fault.

If ^ remains visible as a carrier, check whether the selected mode actually contains an enabled No output association for it. For preceding-vowel classification, both Classify the preceding input and Treat preceding input as a vowel must be on.

For a blank or missing exported font glyph, check the selected components, assigned character address and font chosen in the receiving application. For a failed PNG, reduce the amount of text being exported. For a failed local mode save, export the mode while it is still available in the session.

Changes in this release

Version 0.9.6 adds the included Arabic · Arabizi profile, keeps its digit letters separate from numerals, and gives initial vowels a carrier in backward custom modes. Windows keeps saved modes beside the app when possible; Android keeps them in app storage. The app is named Lambe, with creator attribution to André I. Ben and publisher attribution to Lambe Labs.

Blank startup and mode previews, full-width Windows layout, caret navigation, English combination corrections, final-S curl adjustments, newlines, PNG export, numeral switching, independent mode duplication and No output associations are retained.

Reporting a problem

Include version 0.9.6, Android or Windows, the exact input with its line breaks, the selected mode, what you expected and what appeared. Describe the smallest sequence of actions that reproduces the problem. Add a screenshot or exported image; for a custom mode, include its exported definition. State whether manual movement or font import was involved.

Lambe is an open alpha. Review important transcription yourself and check exported files in the application where you will use them. The About dialog gives app attribution. Created and developed by André I. Ben, Lambe Labs; supplied font and mode material retains its own credits.

Optional reference for exported modes

This reference concerns files that users obtain through Export mode. You can use the graphical editor without reading it. If you edit a definition as text, keep an unchanged copy and import the edited file as a new mode.

The file as a whole

A .tengwar-mode.json file contains one JSON object. JSON uses quoted keys and strings, true or false for switches, square brackets for lists, and no comments or trailing commas. Import mode validates the definition and assigns a new custom identifier.

schemaVersion is 1. It describes the file format, not app version 0.9.6. id begins with custom_; name is the visible mode name. english selects inherited English behaviour when true. autoFlip, autoR, vowelsForward and explicitCarrier correspond to the mode controls; carrier is a recognised base token such as telco.

associations contains the literal input mappings. rules contains contextual output rules. disabledRules records temporarily disabled English groups; removedRules records deleted groups and r_context when contextual R is removed. deletedInputs records removed default inputs.

Copied language modes may also contain referenceData and hebrewDefaults. Preserve these fields when editing an exported copy. They retain inherited behaviour that is not expressed by the ordinary association rows alone.

Association fields

input is a nonempty lowercase literal string without whitespace. token is a recognised glyph name. isMark selects a tehta instead of a base. isVowel supplies R-rule classification. enabled defaults to true. inherited marks an association delegated to the English behaviour; use the graphical editor when changing those defaults.

direction is a keypad anchor from 1 to 9 for a mark association. marks is a list of extra marks attached to a base. Each item has token, direction and dx. dx is a horizontal canvas offset in logical pixels; the graphical editor changes it in 0.2-unit increments. It is not a character-code value.

For noOutput true, token must be empty, isMark must be false and marks must be empty. classifiesPrevious true redirects isVowel classification to the preceding input. This flag is valid only for a No output association.

Rule fields

A rule uses input, scope, context, baseToken, marksOnly, previous, enabled and marks. scope is anywhere, whole, start, end, before or after. before and after require nonempty context. previous or marksOnly requires at least one mark. With both false, baseToken supplies the new base.

Use the exact catalogue names from the app’s pickers. Shape names such as roomen, oore and dotBelow are case-sensitive tokens even though input matching is case-insensitive. The complete example on the next page can be imported as a small instructional mode.

An importable demonstration mode

Copy the complete object below into a plain UTF-8 text file ending .tengwar-mode.json, then use Import mode. It is a small teaching definition, not a full English or Hebrew convention.

```
{
  "schemaVersion": 1,
  "id": "custom_guide_demo",
  "name": "Guide demonstration",
  "english": false,
  "autoFlip": true,
  "autoR": true,
  "vowelsForward": true,
  "explicitCarrier": false,
  "carrier": "telco",
  "disabledRules": [],
  "removedRules": [],
  "deletedInputs": [],
  "associations": [
    {"input":"b", "token":"umbar", "isVowel":false},
    {"input":"m", "token":"malta", "isVowel":false},
    {"input":"r", "token":"oore", "isVowel":false},
    {"input":"a", "token":"a", "isMark":true,
     "isVowel":true, "direction":8},
    {"input":"^", "token":"", "noOutput":true,
     "classifiesPrevious":true, "isVowel":true}
  ],
  "rules": [
    {"input":"m", "scope":"end", "baseToken":"malta",
     "marks":[{"token":"dotBelow", "direction":2, "dx":0}]}
  ]
}
```

Try these inputs

Compare rb with rb^. In rb^ the silent marker classifies b as a vowel for R selection, while the visible b stays umbar. Compare mb with bm: only the final m receives dotBelow. Try ab: the a mark attaches to the following umbar. All other input is intentionally unmapped.

You can build the same definition through Create your own mode using Empty, then add its associations and final-m rule. Export the mode afterwards to see how the graphical choices are represented in a file.